



Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Ψηφιακά Συστήματα VLSI

5η Εργαστηριακή Άσκηση
Υλοποίηση FIR φίλτρου με AXI διεπαφή σε ZYNQ SoC FPGA

ΟΝΟΜΑΤΕΠΩΝΥΜΑ	Ευαγγελία Αγγελάκη (el21037) Μαρία Φατούρου(el21077)
ΕΞΑΜΗΝΟ	8ο
ΟΜΑΔΑ	<u>Βάρδια Β/Ομάδα 17</u>

Στην συγκεκριμένη άσκηση, στόχος μας ήταν να υλοποιήσουμε ένα FIR φίλτρο στην αναπτυξιακή πλακέτα ZYBO. Η βασική υλοποίηση του FIR φίλτρου είναι ίδια με την προηγούμενη εργαστηριακή άσκηση με κάποιες αλλαγές που θα παραθέσουμε παρακάτω. Εν ολίγοις, χωρίζουμε την εργασία μας σε ενότητες : FIR filter , IP address, Block diagram, Vitis

FIR filter

Στη γενική περίπτωση, η σχέση εισόδου- εξόδου ενός FIR φίλτρου είναι η ακόλουθη :

$$y[n] = \sum_{k=0}^M h[k]x[n - k] = h[0]x[n] + h[1]x[n - 1] + \dots + h[M]x[n - M]$$

όπου

- M είναι η τάξη του φίλτρου
- y[n] είναι η έξοδος του φίλτρου τη διακριτή χρονική στιγμή n
- h[k] είναι ο k-οστός συντελεστής του φίλτρου , με k =0,1, 2, ..., M
- x[n] είναι η τιμή του σήματος εισόδου τη διακριτή χρονική στιγμή n

Οι διαφορές που η παρούσα υλοποίηση έχει από την προηγούμενη έγκειται στην απουσία ελεγχόμενου testbench και του ελέγχου των κύκλων μεταξύ των εισόδων. Έτσι, έπρεπε να βρούμε τρόπο να τροποποιήσουμε το κύκλωμά μας για να μην δέχεται προκαθορισμένα είσοδο ανά 8 κύκλους και έτσι να διατηρεί σταθερή τιμή εξόδου μέχρι να ξαναπάρει valid_in. Το valid_out από την στιγμή που τυπώνεται (8 κύκλους μετά το valid_in) θα πρέπει να παραμένει 1 έως ότου να ξαναδεχτεί κάποια είσοδο. Επιπλέον προσθέσαμε enable bit στο mac unit έτσι ώστε όσο περιμένουμε για νέα είσοδο να απενεργοποιείται και να μην συνεχίζει την μέτρηση. Παραθέτουμε πρώτα τους κώδικες που έχουν αλλαγές και μετά και τους υπόλοιπους :

Αλλαγμένοι κώδικες

control unit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity cntrl is
  Port (
    clk : in std_logic;
    reset : in std_logic;
    valid_in : in std_logic;
    rom_addr : out std_logic_vector(2 downto 0);
    ram_addr : out std_logic_vector(2 downto 0);
    mac_init : out std_logic;
    enable, valid_out : out std_logic
  );
```

```
end cntrl;
```

architecture Behavioral of cntrl is

```
signal counter : std_logic_vector(2 downto 0) := "000";
```

```
signal counting : std_logic := '0';
```

```
signal previous_valid : std_logic := '0';
```

```
begin
```

```
process(clk, reset)
```

```
begin
```

```
    if reset = '1' then
```

```
        counter <= (others => '0');
```

```
        counting <= '0';
```

```
        mac_init <= '1';
```

```
        enable <= '0';
```

```
        valid_out <= '0';
```

```
        previous_valid <= '0';
```

```
    elsif rising_edge(clk) then
```

```
        -- Detect rising edge of valid_in
```

```
        if valid_in = '1' and previous_valid = '0' then --change
```

```
            counting <= '1';          -- ksekinaw metrhma
```

```
            counter <= "000";
```

```
            mac_init <= '1';
```

```
            enable <= '1';
```

```
            valid_out <= '0';
```

```
        elsif counting = '1' then
```

```
            mac_init <= '0';
```

```
            if counter = "111" then
```

```
                valid_out <= '1';
```

```
                --enable <= '1';
```

```
                counting <= '0'; -- stop metrhma
```

```
                counter <= "000"; --changed
```

```
            else
```

```
                counter <= counter + 1;
```

```
                enable <= '1';
```

```
                valid_out <= '0';
```

```
            end if;
```

```
        else
```

```
            enable <= '0';
```

```
            valid_out <= '1';
```

```
    end if;
```

```
    -- Store previous value
```

```

        previous_valid <= valid_in;

    end if;
end process;

rom_addr <= counter;
ram_addr <= counter;

end Behavioral;

```

Σχόλια:

Επισημασμένες είναι οι κυριότερες αλλαγές του κώδικα. Έχουμε προσθέσει μια μεταβλητή `previous_valid` με την οποία αποθηκεύουμε σε κάθε κύκλο την τιμή του `valid_in` και στον επόμενο συγκρίνουμε την προηγούμενη με την τωρινή τιμή του `valid_in`. Έτσι ανιχνεύουμε την θετική ακμή του `valid_in` και αυτό το κάνουμε για να ελέγξουμε εάν το `valid_in` διαρκέσει παραπάνω από έναν κύκλο. Επιπλέον έχουμε προσθέσει μία ακόμη μεταβλητή “counting” με την οποία ελέγχουμε πότε θα πρέπει να μετράμε και πότε όχι, και κατ’αυτό τον τρόπο απενεργοποιούμε το “counting” όταν θέλουμε το προγράμμα μας να περιμένει μέχρι να λάβει καινούργια είσοδο.

MAC

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity MAC1 is
    Port (
        enable    : in std_logic;
        clk       : in std_logic;
        mac_init   : in std_logic;
        rom_out    : in std_logic_vector(7 downto 0):= (others => '0');
        ram_out    : in std_logic_vector(7 downto 0):= (others => '0');
        mac_out    : out std_logic_vector(19 downto 0)
    );
end MAC1;

architecture Behavioral of MAC1 is
    signal acc : std_logic_vector(19 downto 0) := (others => '0');
begin
    process(clk,enable)
    begin
        if enable='1' then
            if rising_edge(clk) then
                if mac_init = '1' then
                    acc <= (others => '0');

```

```

        else
            acc <= acc + (rom_out * ram_out);
        end if;
    end if;
else
    acc <= acc;
end if;
end process;
mac_out <= acc;
end Behavioral;

```

Εδώ έχουμε προσθέσει μόνο μία συνθήκη “if enable='1'” και με αυτό τον τρόπο σιγουρεύουμε ότι ο mac θα λειτουργεί μόνο όταν έχουμε ενεργοποιήσει το enable ειδικά θα σταματάει την μέτρηση

Υπόλοιπα components (χωρίς αλλαγές.)

ROM

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity mlab_rom is
    generic (
        coeff_width : integer := 8      --- width of coefficients (bits)
    );
    Port ( clk : in  STD_LOGIC;
          en : in  STD_LOGIC;           --- operation enable
          addr : in  STD_LOGIC_VECTOR (2 downto 0);      -- memory address
          rom_out : out STD_LOGIC_VECTOR (coeff_width-1 downto 0)); -- output data
end mlab_rom;

architecture Behavioral of mlab_rom is

    type rom_type is array (7 downto 0) of std_logic_vector (coeff_width-1 downto 0);
    signal rom : rom_type:= ("00001000", "00000111", "00000110", "00000101",
        "00000100", "00000011", "00000010", "00000001");      -- initialization of rom with
        user data

    signal rdata : std_logic_vector(coeff_width-1 downto 0) := (others => '0');

begin

    rdata <= rom(conv_integer(addr));

    process (clk)
    begin
        if (clk'event and clk = '1') then

```

```

        if (en = '1') then
            rom_out <= rdata;
        end if;
    end if;
end process;
end Behavioral;

```

RAM

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

```

```

entity mlab_ram is
    generic (
        data_width : integer :=8                --- width of data (bits)
    );
    port ( reset : in std_logic;
          clk  : in std_logic;
          we   : in std_logic;                  --- memory write enable
          en   : in std_logic;                  --- operation enable
          addr : in std_logic_vector(2 downto 0); -- memory address
          di   : in std_logic_vector(data_width-1 downto 0); -- input data
          do   : out std_logic_vector(data_width-1 downto 0)); -- output data
end mlab_ram;

```

architecture Behavioral of mlab_ram is

```

    type ram_type is array (7 downto 0) of std_logic_vector (data_width-1 downto 0);
    signal RAM : ram_type := (others => (others => '0'));

```

begin

```

    process (clk,reset)
    begin
        if reset = '1' then
            for i in 0 to 7 loop
                RAM(i) <= "00000000";
            end loop;
        elsif clk'event and clk = '1' then
            if en = '1' then
                if we = '1' then                -- write operation
                    for i in 6 downto 0 loop
                        RAM(i+1) <= RAM(i);
                    end loop;
                    RAM(0) <= di;
                elsif we = '0' then              -- read operation
                    do <= RAM( conv_integer(addr));
                end if;
            end if;
        end if;
    end process;

```

```

        end if;
    end if;
end if;
end process;
end Behavioral;

```

D flip-flop

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

entity dffone is

```

    Port (
        clk : in std_logic;
        reset : in std_logic;
        D : in std_logic;
        Q : out std_logic
    );
end dffone;

```

architecture Behavioral of dffone is

```

begin
    process(clk, reset)
    begin
        if reset = '1' then
            Q <= '0';
        elsif rising_edge(clk) then
            Q <= D;
        end if;
    end process;
end Behavioral;

```

FIR

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

```

entity FIR is

```

    Port (clk : in std_logic;
        rst : in std_logic;
        valid_in : in std_logic;
        x : in std_logic_vector(7 downto 0);
        valid_out : out std_logic;
        y : out std_logic_vector(19 downto 0)
    );
end FIR;

```

architecture Structural of FIR is

component cntrl

```
Port ( clk : in std_logic;
reset : in std_logic;
valid_in : in std_logic;
rom_addr : out std_logic_vector(2 downto 0);
ram_addr : out std_logic_vector(2 downto 0);
mac_init : out std_logic;
enable, valid_out : out std_logic);
end component;
```

component mac1

```
Port (
    enable : in std_logic;
    clk : in std_logic;
    --reset : in std_logic;
    mac_init : in std_logic;
    rom_out : in std_logic_vector(7 downto 0);
    ram_out : in std_logic_vector(7 downto 0);
    mac_out : out std_logic_vector(19 downto 0)
);
end component;
```

component mlab_ram

```
generic (
    data_width : integer := 8
);
port (
    reset : in std_logic;
    clk : in std_logic;
    we : in std_logic;
    en : in std_logic;
    addr : in std_logic_vector(2 downto 0);
    di : in std_logic_vector(data_width-1 downto 0);
    do : out std_logic_vector(data_width-1 downto 0)
);
end component;
```

component mlab_rom

```
generic (
    coeff_width : integer := 8
)
--- width of coefficients
(bits)
);
Port ( clk : in STD_LOGIC;
    en : in STD_LOGIC;
    --- operation enable
```

```

        addr : in STD_LOGIC_VECTOR (2 downto 0);                -- memory
address
        rom_out : out STD_LOGIC_VECTOR (coeff_width-1 downto 0)); -- output data
end component;

component dffone
port (clk : in std_logic;
      reset : in std_logic;
      D : in std_logic;
      Q : out std_logic
    );
end component;

signal rom_addrs : std_logic_vector(2 downto 0);
signal ram_addrs : std_logic_vector(2 downto 0);
signal macinit : std_logic;
signal enable : std_logic;
signal ram_out : std_logic_vector(7 downto 0);
signal rom_out : std_logic_vector(7 downto 0);
signal valid_out1 : std_logic;

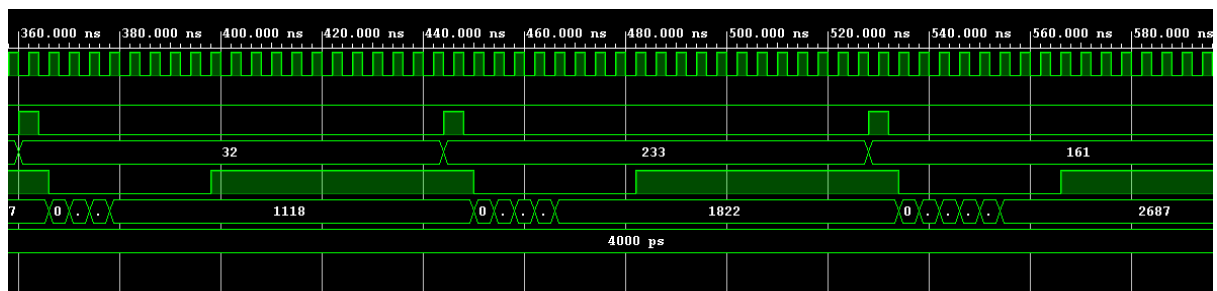
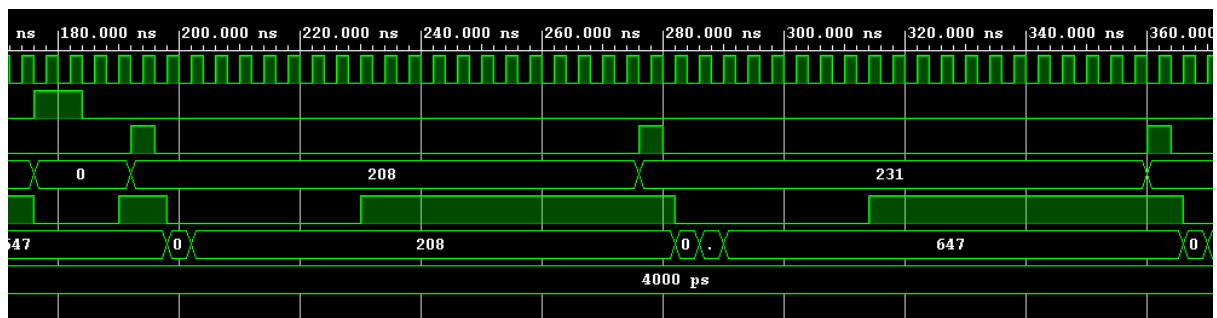
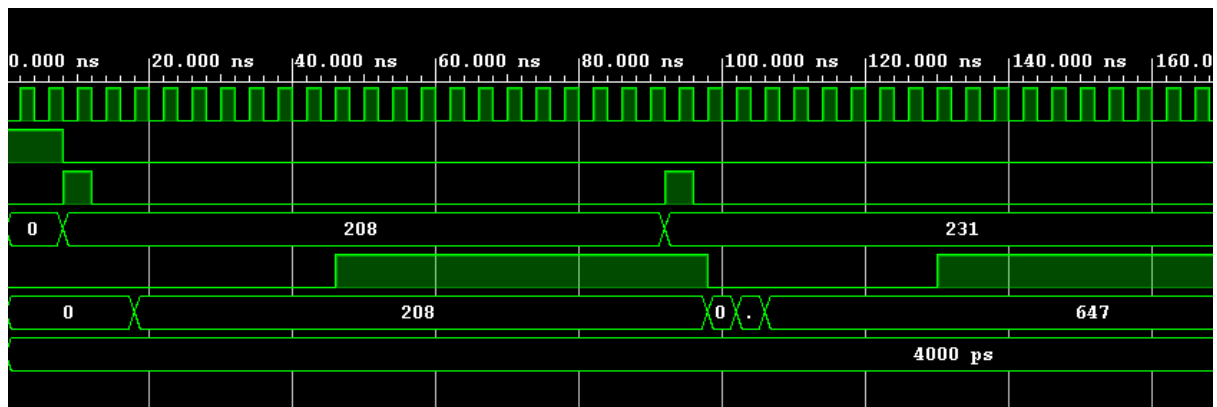
begin

CU : cntrl port map(clk,rst,valid_in,rom_addrs,ram_addrs,macinit,enable,valid_out1); --chng
dff2 : dffone port map(clk,rst,valid_out1,valid_out);
ramunit : mlab_ram port map(rst,clk,valid_in,'1',ram_addrs,x,ram_out);
romunit : mlab_rom port map(clk,'1',rom_addrs,rom_out);
macunit : mac1 port map(enable,clk,macinit,rom_out,ram_out,y);
end Structural;

```

Σχόλια:

Τρέξαμε σε simulation το fir filter για να επιβεβαιώσουμε την λειτουργία του πριν το ενσωματώσουμε στο zybo και όπως παρατηρούμε η λειτουργία του κυκλώματος είναι η επιθυμητή. Έχουμε ρυθμίσει το testbench έτσι ώστε να έρχεται valid_in ανά 20 κύκλους για να ελέγξουμε ότι το valid_out εμφανίζεται 8 κύκλους μετά από το valid_in και μετά παραμένει ενεργοποιημένο έως ότου επέλθει το επόμενο valid_in. Σε αυτή την αναμονή ο mac διατηρεί την τιμή του έτσι ώστε να έχουμε τα σωστά αποτελέσματα.



IP Address

Τώρα θα επισημάνουμε τα βήματα που ακολουθήσαμε ώστε να υλοποιήσουμε το FIR στο ZYBO και θα ασχοληθούμε με την IP address. Η επικοινωνία μεταξύ PS και PL πραγματοποιείται μέσω ενός διαύλου επικοινωνίας εύρους 32 bit, συνεπώς η αποστολή των σημάτων εισόδου και εγκυρότητας θα συμβαίνει ταυτόχρονα στην ίδια λέξη των 32 bit. Έτσι λοιπόν δημιουργήσαμε ένα νέο Project στο Vivado, επιλέγουμε Tools, Create and Package New IP. Διαλέγουμε το AXI Peripheral και πατάμε next ώστε να δημιουργηθεί το IP Package. Στη συνέχεια προσθέτουμε τα αρχεία sources του τροποποιημένου φίλτρου FIR και δημιουργείται το αρχείο με κατάληξη _v1_0_s00_AXI του οποίου το περιεχόμενο τροποποιούμε. Παραθέτουμε παρακάτω τον κώδικα:

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity my_ip_v1_0_S00_AXI is
    generic (
        -- Users to add parameters here

        -- User parameters ends
        -- Do not modify the parameters beyond this line

        -- Width of S_AXI data bus
        C_S_AXI_DATA_WIDTH  : integer      := 32;
        -- Width of S_AXI address bus
        C_S_AXI_ADDR_WIDTH  : integer      := 4
    );
    port (
        -- Users to add ports here

        -- User ports ends
        -- Do not modify the ports beyond this line

        -- Global Clock Signal
        S_AXI_ACLK           : in std_logic;
        -- Global Reset Signal. This Signal is Active LOW
        S_AXI_ARESETN        : in std_logic;
        -- Write address (issued by master, accepted by Slave)
        S_AXI_AWADDR         : in  std_logic_vector(C_S_AXI_ADDR_WIDTH-1
downto 0);
        -- Write channel Protection type. This signal indicates the
        -- privilege and security level of the transaction, and whether
        -- the transaction is a data access or an instruction access.
        S_AXI_AWPROT         : in std_logic_vector(2 downto 0);
        -- Write address valid. This signal indicates that the master signaling
        -- valid write address and control information.
        S_AXI_AWVALID        : in std_logic;
        -- Write address ready. This signal indicates that the slave is ready
        -- to accept an address and associated control signals.
        S_AXI_AWREADY        : out std_logic;
        -- Write data (issued by master, accepted by Slave)
        S_AXI_WDATA          : in  std_logic_vector(C_S_AXI_DATA_WIDTH-1
downto 0);
        -- Write strobes. This signal indicates which byte lanes hold
        -- valid data. There is one write strobe bit for each eight

```

```

-- bits of the write data bus.
S_AXI_WSTRB      : in std_logic_vector((C_S_AXI_DATA_WIDTH/8)-1
downto 0);
-- Write valid. This signal indicates that valid write
-- data and strobes are available.
S_AXI_WVALID     : in std_logic;
-- Write ready. This signal indicates that the slave
-- can accept the write data.
S_AXI_WREADY     : out std_logic;
-- Write response. This signal indicates the status
-- of the write transaction.
S_AXI_BRESP      : out std_logic_vector(1 downto 0);
-- Write response valid. This signal indicates that the channel
-- is signaling a valid write response.
S_AXI_BVALID     : out std_logic;
-- Response ready. This signal indicates that the master
-- can accept a write response.
S_AXI_BREADY     : in std_logic;
-- Read address (issued by master, accepted by Slave)
S_AXI_ARADDR     : in  std_logic_vector(C_S_AXI_ADDR_WIDTH-1
downto 0);
-- Protection type. This signal indicates the privilege
-- and security level of the transaction, and whether the
-- transaction is a data access or an instruction access.
S_AXI_ARPROT     : in std_logic_vector(2 downto 0);
-- Read address valid. This signal indicates that the channel
-- is signaling valid read address and control information.
S_AXI_ARVALID    : in std_logic;
-- Read address ready. This signal indicates that the slave is
-- ready to accept an address and associated control signals.
S_AXI_ARREADY    : out std_logic;
-- Read data (issued by slave)
S_AXI_RDATA      : out  std_logic_vector(C_S_AXI_DATA_WIDTH-1
downto 0);
-- Read response. This signal indicates the status of the
-- read transfer.
S_AXI_RRESP      : out std_logic_vector(1 downto 0);
-- Read valid. This signal indicates that the channel is
-- signaling the required read data.
S_AXI_RVALID     : out std_logic;
-- Read ready. This signal indicates that the master can
-- accept the read data and response information.
S_AXI_RREADY     : in std_logic
);

```

```
end my_ip_v1_0_S00_AXI;
```

architecture arch_imp of my_ip_v1_0_S00_AXI is

```
-- AXI4LITE signals
```

```
signal axi_awaddr   : std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
signal axi_awready  : std_logic;
signal axi_wready   : std_logic;
signal axi_bresp    : std_logic_vector(1 downto 0);
signal axi_bvalid   : std_logic;
signal axi_araddr   : std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
signal axi_arready  : std_logic;
signal axi_rdata    : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
signal axi_rresp    : std_logic_vector(1 downto 0);
signal axi_rvalid   : std_logic;
```

```
-- Example-specific design signals
```

```
-- local parameter for addressing 32 bit / 64 bit C_S_AXI_DATA_WIDTH
```

```
-- ADDR_LSB is used for addressing 32/64 bit registers/memories
```

```
-- ADDR_LSB = 2 for 32 bits (n downto 2)
```

```
-- ADDR_LSB = 3 for 64 bits (n downto 3)
```

```
constant ADDR_LSB : integer := (C_S_AXI_DATA_WIDTH/32)+ 1;
```

```
constant OPT_MEM_ADDR_BITS : integer := 1;
```

```
-----
---- Signals for user logic register space example
-----
```

```
---- Number of Slave Registers 4
```

```
signal slv_reg0      :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
signal slv_reg1      :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
signal slv_reg2      :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
signal slv_reg3      :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
signal slv_reg_rden  : std_logic;
signal slv_reg_wren  : std_logic;
signal reg_data_out  :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
signal byte_index    : integer;
signal aw_en         : std_logic;
```

component FIR is

Port (clk : in std_logic;

rst : in std_logic;

valid_in : in std_logic;

x : in std_logic_vector(7 downto 0);

valid_out : out std_logic;

y : out std_logic_vector(19 downto 0)

```
);  
end component;
```

```
begin
```

```
-- I/O Connections assignments
```

```
S_AXI_AWREADY <= axi_awready;  
S_AXI_WREADY  <= axi_wready;  
S_AXI_BRESP   <= axi_bresp;  
S_AXI_BVALID  <= axi_bvalid;  
S_AXI_ARREADY <= axi_arready;  
S_AXI_RDATA   <= axi_rdata;  
S_AXI_RRESP   <= axi_rresp;  
S_AXI_RVALID  <= axi_rvalid;  
-- Implement axi_awready generation  
-- axi_awready is asserted for one S_AXI_ACLK clock cycle when both  
-- S_AXI_AWVALID and S_AXI_WVALID are asserted. axi_awready is  
-- de-asserted when reset is low.
```

```
process (S_AXI_ACLK)
```

```
begin
```

```
if rising_edge(S_AXI_ACLK) then
```

```
if S_AXI_ARESETN = '0' then
```

```
axi_awready <= '0';
```

```
aw_en <= '1';
```

```
else
```

```
if (axi_awready = '0' and S_AXI_AWVALID = '1' and S_AXI_WVALID = '1'  
and aw_en = '1') then
```

```
-- slave is ready to accept write address when
```

```
-- there is a valid write address and write data
```

```
-- on the write address and data bus. This design
```

```
-- expects no outstanding transactions.
```

```
axi_awready <= '1';
```

```
-- aw_en <= '0';
```

```
elsif (S_AXI_BREADY = '1' and axi_bvalid = '1') then
```

```
aw_en <= '1';
```

```
axi_awready <= '0';
```

```
else
```

```
axi_awready <= '0';
```

```
end if;
```

```
end if;
```

```
end if;
```

```
end process;
```

-- Implement axi_awaddr latching
-- This process is used to latch the address when both
-- S_AXI_AWVALID and S_AXI_WVALID are valid.

```
process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_awaddr <= (others => '0');
        else
            if (axi_awready = '0' and S_AXI_AWVALID = '1' and S_AXI_WVALID = '1'
and aw_en = '1') then
                -- Write Address latching
                axi_awaddr <= S_AXI_AWADDR;
            end if;
        end if;
    end if;
end process;
```

-- Implement axi_wready generation
-- axi_wready is asserted for one S_AXI_ACLK clock cycle when both
-- S_AXI_AWVALID and S_AXI_WVALID are asserted. axi_wready is
-- de-asserted when reset is low.

```
process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_wready <= '0';
        else
            if (axi_wready = '0' and S_AXI_WVALID = '1' and S_AXI_AWVALID = '1'
and aw_en = '1') then
                -- slave is ready to accept write data when
                -- there is a valid write address and write data
                -- on the write address and data bus. This design
                -- expects no outstanding transactions.
                axi_wready <= '1';
            else
                axi_wready <= '0';
            end if;
        end if;
    end if;
end process;
```

```

-- Implement memory mapped register select and write logic generation
-- The write data is accepted and written to memory mapped registers when
-- axi_awready, S_AXI_WVALID, axi_wready and S_AXI_WVALID are
asserted. Write strobes are used to
-- select byte enables of slave registers while writing.
-- These registers are cleared when reset (active low) is applied.
-- Slave register write enable is asserted when valid address and data are
available
-- and the slave is ready to accept the write address and write data.
slv_reg_wren <= axi_wready and S_AXI_WVALID and axi_awready and
S_AXI_AWVALID ;

```

```

process (S_AXI_ACLK)
variable loc_addr :std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
begin
if rising_edge(S_AXI_ACLK) then
if S_AXI_ARESETN = '0' then
slv_reg0 <= (others => '0');
-- slv_reg1 <= (others => '0');
slv_reg2 <= (others => '0');
slv_reg3 <= (others => '0');
else
loc_addr := axi_awaddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto
ADDR_LSB);
if (slv_reg_wren = '1') then
case loc_addr is
when b"00" =>
for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
if ( S_AXI_WSTRB(byte_index) = '1' ) then
-- Respective byte enables are asserted as per write strobes
-- slave register 0
slv_reg0(byte_index*8+7 downto byte_index*8) <=
S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
end if;
end loop;
-- when b"01" =>
-- for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
-- if ( S_AXI_WSTRB(byte_index) = '1' ) then
-- Respective byte enables are asserted as per write strobes
-- slave register 1
-- slv_reg1(byte_index*8+7 downto byte_index*8) <=
S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
-- end if;
-- end loop;

```

```

when b"10" =>
  for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
    if ( S_AXI_WSTRB(byte_index) = '1' ) then
      -- Respective byte enables are asserted as per write strobes
      -- slave register 2
      slv_reg2(byte_index*8+7 downto byte_index*8) <=
S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
    end if;
  end loop;
when b"11" =>
  for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
    if ( S_AXI_WSTRB(byte_index) = '1' ) then
      -- Respective byte enables are asserted as per write strobes
      -- slave register 3
      slv_reg3(byte_index*8+7 downto byte_index*8) <=
S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
    end if;
  end loop;
when others =>
  slv_reg0 <= slv_reg0;
  slv_reg1 <= slv_reg1;
  slv_reg2 <= slv_reg2;
  slv_reg3 <= slv_reg3;
end case;
else slv_reg0(8)<='0';
end if;
end if;
end if;
end process;

```

-- Implement write response logic generation
 -- The write response and response valid signals are asserted by the slave
 -- when axi_wready, S_AXI_WVALID, axi_wready and S_AXI_WVALID are asserted.

-- This marks the acceptance of address and indicates the status of
 -- write transaction.

```

process (S_AXI_ACLK)
begin
  if rising_edge(S_AXI_ACLK) then
    if S_AXI_ARESETN = '0' then
      axi_bvalid <= '0';
      axi_bresp <= "00"; --need to work more on the responses
    else

```

```

        if (axi_awready = '1' and S_AXI_AWVALID = '1' and axi_wready = '1' and
S_AXI_WVALID = '1' and axi_bvalid = '0' ) then
            axi_bvalid <= '1';
            axi_bresp <= "00";
            elsif (S_AXI_BREADY = '1' and axi_bvalid = '1') then --check if bready is
asserted while bvalid is high)
                axi_bvalid <= '0'; -- (there is a possibility that bready
is always asserted high)
            end if;
        end if;
    end if;
end process;

```

```

-- Implement axi_arready generation
-- axi_arready is asserted for one S_AXI_ACLK clock cycle when
-- S_AXI_ARVALID is asserted. axi_arready is
-- de-asserted when reset (active low) is asserted.
-- The read address is also latched when S_AXI_ARVALID is
-- asserted. axi_araddr is reset to zero on reset assertion.

```

```

process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_arready <= '0';
            axi_araddr <= (others => '1');
        else
            if (axi_arready = '0' and S_AXI_ARVALID = '1') then
                -- indicates that the slave has accepted the valid read address
                axi_arready <= '1';
                -- Read Address latching
                axi_araddr <= S_AXI_ARADDR;
            else
                axi_arready <= '0';
            end if;
        end if;
    end if;
end process;

```

```

-- Implement axi_arvalid generation
-- axi_rvalid is asserted for one S_AXI_ACLK clock cycle when both
-- S_AXI_ARVALID and axi_arready are asserted. The slave registers
-- data are available on the axi_rdata bus at this instance. The
-- assertion of axi_rvalid marks the validity of read data on the

```

```

-- bus and axi_rresp indicates the status of read transaction.axi_rvalid
-- is deasserted on reset (active low). axi_rresp and axi_rdata are
-- cleared to zero on reset (active low).
process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_rvalid <= '0';
            axi_rresp <= "00";
        else
            if (axi_arready = '1' and S_AXI_ARVALID = '1' and axi_rvalid = '0') then
                -- Valid read data is available at the read data bus
                axi_rvalid <= '1';
                axi_rresp <= "00"; -- 'OKAY' response
            elsif (axi_rvalid = '1' and S_AXI_RREADY = '1') then
                -- Read data is accepted by the master
                axi_rvalid <= '0';
            end if;
        end if;
    end if;
end process;

```

```

-- Implement memory mapped register select and read logic generation
-- Slave register read enable is asserted when valid address is available
-- and the slave is ready to accept the read address.
slv_reg_rden <= axi_arready and S_AXI_ARVALID and (not axi_rvalid) ;

```

```

process (slv_reg0, slv_reg1, slv_reg2, slv_reg3, axi_araddr,
S_AXI_ARESETN, slv_reg_rden)
variable loc_addr :std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
begin
    -- Address decoding for reading registers
    loc_addr := axi_araddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto
ADDR_LSB);
    case loc_addr is
        when b"00" =>
            reg_data_out <= slv_reg0;
        when b"01" =>
            reg_data_out <= slv_reg1;
        when b"10" =>
            reg_data_out <= slv_reg2;
        when b"11" =>
            reg_data_out <= slv_reg3;
        when others =>

```

```

        reg_data_out <= (others => '0');
    end case;
end process;

-- Output register or memory read data
process( S_AXI_ACLK ) is
begin
    if (rising_edge (S_AXI_ACLK)) then
        if ( S_AXI_ARESETN = '0' ) then
            axi_rdata <= (others => '0');
        else
            if (slv_reg_rden = '1') then
                -- When there is a valid read address (S_AXI_ARVALID) with
                -- acceptance of read address by the slave (axi_arready),
                -- output the read data
                -- Read address mux
                axi_rdata <= reg_data_out;    -- register read data
            end if;
        end if;
    end if;
end process;

-- Add user logic here
FIR1 : FIR port map (
    clk=>S_AXI_ACLK,
    rst=> slv_reg0(9),
    valid_in=>slv_reg0(8),
    y => slv_reg1(19 downto 0),
    valid_out =>slv_reg1(20) ,    --arm reads from slv0 - the ip writes to it
    x=>slv_reg0(7 downto 0)
);
end arch_imp;

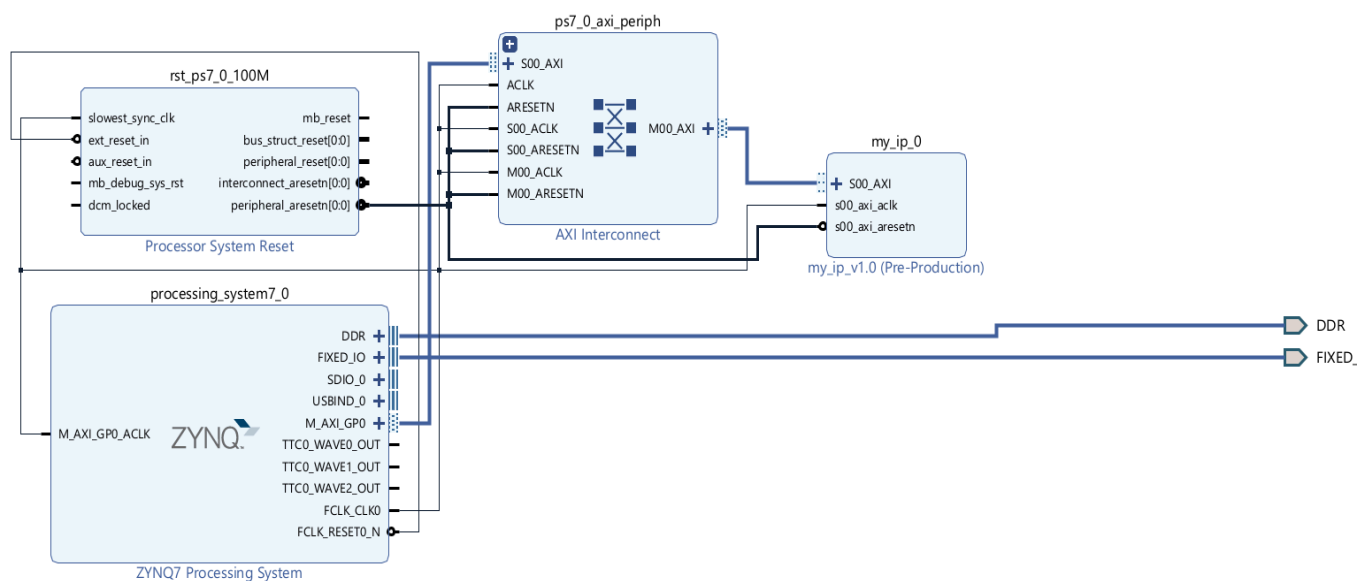
```

Σχόλια:

Στην αρχή του αρχείου περιγράφεται η εσωτερική αρχιτεκτονική του AXI slave module όπου ορίζουμε τα σήματα AXI και προσθέτουμε το component του φίλτρου FIR που δημιουργήσαμε. Έχουμε στη διάθεσή μας τέσσερις καταχωρητές των 32 bit, slv_reg0, slv_reg1, slv_reg2, slv_reg3. Στη συνέχεια υλοποιούνται διάφορα process blocks τα οποία χειρίζονται την επικοινωνία με τον AXI. Διαπιστώνουμε ότι χρειάζεται να σχολιάσουμε ένα μεγάλο κομμάτι του κώδικα, καθώς όταν το PL χρησιμοποιεί έναν register για εγγραφή πρέπει να αποτρέψουμε στον arm να πραγματοποιήσει εγγραφή στον ίδιο register και το αντίστροφο. Με αυτόν τον τρόπο εγγυόμαστε ότι τα δεδομένα στον καταχωρητή θα παραμείνουν αναλλοίωτα. Επίσης χρειάζεται να προσθέσουμε τη συνθήκη if όταν είναι

δηλαδή έτοιμο να διαβάσει από το software ώστε να κάνουμε το valid_in=0. Αυτό συμβαίνει ώστε το FIR να μην διαβάζει πολλές φορές την ίδια είσοδο. Τέλος γράφουμε το user logic που απλά αποτελεί ένα port map των στοιχείων του FIR με τους registers του AXI. Παραπάνω έχουμε επισημάνει τα σημεία του κώδικα που υπέστησαν αλλαγές.

Block diagram



Όπως φαίνεται παραπάνω, για το block diagram χρειαζόμαστε το processing system του zynq καθώς και το processor system reset component. Αυτά επικοινωνούν με την ip (το programmable logic που περιλαμβάνει και την λογική μας) διαμέσου της διεπαφής axi όπως φαίνεται παραπάνω. Το πρωτόκολλο AXI4-Lite χρησιμεύει ως τρόπος επικοινωνίας μεταξύ του PL και του PS στα FPGA αν και παρατηρούμε ότι είναι σχετικά πιο αργό από άλλα πρωτόκολλα καθώς επιβάλλει ένα έξτρα στάδιο μεταξύ του ενός και του άλλου και κατ επέκταση και καθυστέρηση.

Vitis

```
#include <stdio.h>
#include "platform.h"
#include "xparameters.h"
#include "xil_io.h"
#include "xil_printf.h"
#include "sleep.h"

#define MYIP_BASEADDR    XPAR_MYFRESHIP_0_S00_AXI_BASEADDR

#define REG_INPUT_ADDR   (MYIP_BASEADDR + 0x00) // slv_reg0
#define REG_OUTPUT_ADDR  (MYIP_BASEADDR + 0x04) // slv_reg1

#define VALID_OUT_MASK   0x00100000 // Bit 20
#define Y_MASK           0x000FFFFFF // Bits 0-19

int main() {
    init_platform();

    u32 x, rst, valid_in;
    u32 input_data, output_data, output_reg;
    int retry;

    while (1) {
        xil_printf("\n=== new input in FIR filter ===\n");

        xil_printf("give value x (8 bits): ");
        scanf("%lu", &x);

        xil_printf("give reset (0 ή 1): ");
        scanf("%lu", &rst);

        xil_printf("give valid_in (0 ή 1): ");
        scanf("%lu", &valid_in);

        input_data = (x & 0xFF) | ((valid_in & 0x1) << 8) | ((rst & 0x1) << 9);

        Xil_Out32(REG_INPUT_ADDR, input_data);

        xil_printf("wait for valid output...\n");

        retry = 0;
        while (1) {
            output_reg = Xil_In32(REG_OUTPUT_ADDR);

            if ((output_reg & VALID_OUT_MASK) != 0) {
                output_data = output_reg & Y_MASK;
```

```

        xil_printf("output y = %lu (0x%05lx)\n", output_data, output_data);
        break;
    }
    retry++;
    if (retry > 1000000) {
        xil_printf("Timeout:no valid output\n");
        break;
    }
}
}
cleanup_platform();
return 0;
}

```

Σχόλια:

Αρχικά ορίζουμε την ip μας ως MYIP_BASEADDR και ορίζουμε τον slv_reg0 ως REG_INPUT_ADDR και τον slv_reg1 ως REG_OUTPUT_ADDR. Στη συνέχεια δίνουμε την είσοδο στον arm και στη συνέχεια τοποθετούμε όλο το input σε μία μεταβλητή input_data. Όταν το output είναι διαφορετικό του μηδενός και το valid_out είναι ίσο με 1 εμφανίζουμε στην έξοδο το αποτέλεσμα y. Εάν αυτός ο έλεγχος πραγματοποιηθεί περισσότερες από 1000000 φορές στην έξοδο τυπώνεται "no valid output". Με αυτόν τον τρόπο βλέπουμε τα αποτελέσματα στο serial monitor του ZYBO.